

PYTHON STANDARD LIBRARY

Six Essential Modules

A quick, practical guide to csv, json, random, time, datetime, and pathlib — what they do, their key functions, and how to use them.

csv

json

random

time

datetime

pathlib

The **csv** module reads and writes CSV (comma-separated values) files — the plain-text spreadsheet format used everywhere for tabular data exports. It handles quoting, delimiters, and edge cases so you don't have to split strings by hand.

KEY FUNCTIONS

<code>csv.reader(f)</code>	Reads rows from a file as lists of strings.
<code>csv.writer(f)</code>	Writes rows (lists) to a file as CSV text.
<code>csv.DictReader(f)</code>	Reads rows as dictionaries, using the header row as keys.
<code>csv.DictWriter(f, fieldnames)</code>	Writes dictionaries as CSV rows in a defined column order.

EXAMPLE

```
# reading a CSV file
import csv

with open("people.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["age"])

# writing a CSV file
with open("out.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "age"])
    writer.writerow(["Alice", 30])
```

Use it for: importing/exporting spreadsheet data, logging structured records, or moving data between tools like Excel and Python.



The **json** module converts between Python objects (dicts, lists, strings, numbers) and JSON text — the standard format for APIs, config files, and data interchange on the web.

KEY FUNCTIONS

<code>json.dumps(obj)</code>	Converts a Python object to a JSON-formatted string.
<code>json.loads(s)</code>	Parses a JSON string into a Python object.
<code>json.dump(obj, f)</code>	Writes a Python object as JSON directly to a file.
<code>json.load(f)</code>	Reads JSON from a file and parses it into a Python object.

EXAMPLE

```
import json

data = {"name": "Alice", "age": 30}

# Python object -> JSON string
text = json.dumps(data, indent=2)

# JSON string -> Python object
parsed = json.loads(text)

# write / read JSON files
with open("data.json", "w") as f:
    json.dump(data, f)
```

Use it for: talking to web APIs, saving config or settings files, and serializing structured data in a human-readable format.

The **random** module generates pseudo-random numbers and makes random selections — useful for simulations, games, sampling, and shuffling data.

KEY FUNCTIONS

<code>random.random()</code>	Returns a random float between 0.0 and 1.0.
<code>random.randint(a, b)</code>	Returns a random integer between a and b (inclusive).
<code>random.choice(seq)</code>	Returns a random element from a sequence.
<code>random.shuffle(list)</code>	Shuffles a list in place, randomizing its order.
<code>random.sample(seq, k)</code>	Returns k unique random elements from a sequence.

EXAMPLE

```
import random

dice_roll = random.randint(1, 6)
winner = random.choice(["Alice", "Bob", "Cara"])

cards = ["A", "K", "Q", "J"]
random.shuffle(cards)

sample = random.sample(range(100), 5)
```

Use it for: games, shuffling quiz questions, randomized testing data, or simple simulations. Not suitable for cryptography — use the **secrets** module for that.

The **time** module works with low-level time functions — measuring elapsed time, pausing execution, and reading the system clock.

KEY FUNCTIONS

<code>time.time()</code>	Returns the current time in seconds since the Unix epoch.
<code>time.sleep(s)</code>	Pauses program execution for <i>s</i> seconds.
<code>time.perf_counter()</code>	Returns a high-precision timer, ideal for measuring code duration.
<code>time.strftime(fmt)</code>	Formats the current time as a string.

EXAMPLE

```
import time

start = time.perf_counter()
time.sleep(1)  # pause for 1 second
elapsed = time.perf_counter() - start

print(f"Took {elapsed:.2f} seconds")
print(time.strftime("%Y-%m-%d %H:%M:%S"))
```

Use it for: benchmarking code, adding delays, and timestamping events at the system level.

The **datetime** module provides classes for working with calendar dates and times — creating, formatting, comparing, and doing arithmetic on dates in a much more readable way than **time**.

KEY FUNCTIONS & CLASSES

<code>datetime.now()</code>	Returns the current local date and time.
<code>datetime.date(y, m, d)</code>	Creates a date object for a specific calendar day.
<code>datetime.strftime(fmt)</code>	Formats a datetime object as a string.
<code>datetime.strptime(s, fmt)</code>	Parses a string into a datetime object.
<code>timedelta(days=n)</code>	Represents a duration, used for date arithmetic.

EXAMPLE

```
from datetime import datetime, timedelta

now = datetime.now()
birthday = datetime(2000, 5, 17)

# date arithmetic
next_week = now + timedelta(days=7)

# formatting and parsing
text = now.strftime("%B %d, %Y")
parsed = datetime.strptime("2026-09-09", "%Y-%m-%d")
```

Use it for: scheduling, age/duration calculations, formatting human-readable dates, and parsing date strings from files or APIs.

The **pathlib** module offers an object-oriented way to handle filesystem paths — a modern, cross-platform replacement for string-based paths and the older **os.path** module.

KEY FUNCTIONS & METHODS

<code>Path("x")</code>	Creates a path object representing a file or folder.
<code>Path.exists()</code>	Checks whether the path exists on disk.
<code>Path.mkdir()</code>	Creates a new directory at the path.
<code>Path.glob(pattern)</code>	Finds files matching a pattern, e.g. "*.txt".
<code>Path.read_text()</code> / <code>write_text()</code>	Reads or writes a file's text content directly.

EXAMPLE

```
from pathlib import Path

folder = Path("documents")
folder.mkdir(exist_ok=True)

file = folder / "notes.txt"
file.write_text("Hello, pathlib!")

# list all .txt files
for txt_file in folder.glob("*.txt"):
    print(txt_file.name)
```

Use it for: building, checking, and manipulating file paths in a readable, cross-platform way — joining paths with "/" instead of manual string concatenation.